JAVASCRIPT • MODULE 02

Functions & Scope

Master the Building Blocks of Every JS Program

■ FUNCTIONS

■ LEARNING OUTCOME

By the end of this module you will write all types of JavaScript functions, understand closure and lexical scope, use higher-order functions (map/filter/reduce), and know how 'this' works.

■ Skills You Will Gain

- Write function declarations, expressions, and arrow functions
- Use default params, rest params, and spread operator
- Understand lexical scope, hoisting, and scope chain
- Create closures for private state
- Use higher-order functions: map, filter, reduce
- Understand the 'this' keyword

■ Functions are the building blocks of JavaScript. Every design pattern, every framework, every program is built from functions. Master them and everything clicks.

SECTION 1

Function Types

```
// 1. Function Declaration -- hoisted (callable before definition) function greet(name) { return `Hello, ${name}!`; } // 2. Function Expression -- NOT hoisted const multiply = function(a, b) { return a * b; }; // 3. Arrow Function -- concise, no own 'this' const add = (a, b) => a + b; // implicit return const square = n => n * n; // single param: no () needed const sayHi = () => 'Hello!'; // no params: () required const getUser = id => ({ id, active: true }); // return object: wrap in () // 4. IIFE -- Immediately Invoked Function Expression (function() { const privateVar = 'only here'; console.log('Runs immediately!'); })(); // When to use arrow vs regular function: // Arrow: callbacks, array methods, inside class methods (inherit this) // Regular: object methods, constructors, when you need own 'this'
```

SECTION 2

Parameters: Default, Rest, Spread

```
// DEFAULT PARAMETERS function createUser(name, role = 'viewer', active = true) { return { name, role, active }; // shorthand property names } createUser('Alice'); // { name:'Alice', role:'viewer', active:true } createUser('Bob', 'admin'); // { name:'Bob', role:'admin', active:true } // REST PARAMETERS -- collects remaining args into array function sum(...numbers) { return numbers.reduce((total, n) => total + n, 0); } sum(1, 2, 3, 4, 5) // 15 // SPREAD -- expands array/object into individual elements const nums = [3, 1, 4, 1, 5]; Math.max(...nums) // 5 const copy = [...nums]; // shallow array copy const merged = [...nums, 6, 7]; // extend array // Object spread -- merge and clone const defaults = { theme:'light', lang:'en', fontSize:14 }; const prefs = { theme:'dark', fontSize:16 }; const settings = { ...defaults, ...prefs }; // later props win // { theme:'dark', lang:'en', fontSize:16 } // DESTRUCTURING in parameters function show({ name, age, city = 'Unknown' }) { console.log(`${name} (${age}) from ${city}`); } show({ name:'Alice', age:28, city:'Mumbai' }); // Array destructuring in params function firstAndRest([first, ...rest]) { return { first, rest }; } firstAndRest([1,2,3,4]); // { first:1, rest:[2,3,4] }
```

SECTION 3

Scope, Closures & Higher-Order Functions

Closure -- Inner function remembers outer scope after outer returns

When a function is created inside another function, it CLOSES OVER the outer variables. Even after the outer function returns, the inner function still has access to those variables. Use cases: private state, function factories, event handlers with shared data.

```
// Closure example: private counter function makeCounter(start = 0) { let count = start; // private -- NOT accessible from outside return { increment: () => ++count, decrement: () => --count, getValue: () => count }; } const counter = makeCounter(10); counter.increment(); // 11 counter.increment(); // 12 // count -- undefined outside -- TRULY PRIVATE via closure // Function factory using closure function multiplier(factor) { return number => number * factor; // 'factor' remembered } const double = multiplier(2); const triple = multiplier(3); double(5) // 10 triple(5) // 15 // HIGHER-ORDER FUNCTIONS const numbers = [1,2,3,4,5,6,7,8,9,10]; // .map() -- transform every element, returns NEW array const doubled = numbers.map(n => n * 2); // [2,4,6,8,10,12,14,16,18,20] // .filter() -- keep elements where function returns true const evens = numbers.filter(n => n % 2 === 0); // [2,4,6,8,10] // .reduce() -- collapse to single value const sum = numbers.reduce((acc, n) => acc + n, 0); // 55 // CHAINING -- filter -> map -> reduce const result = numbers .filter(n => n % 2 === 0) // evens: [2,4,6,8,10] .map(n => n ** 2) // squared: [4,16,36,64,100] .reduce((s, n) => s + n, 0); // total: 220
```

■ PRACTICE Q & A

Q1. What is the difference between a function declaration and expression?

A: *Declaration (function foo(){}) is hoisted -- callable before definition. Expression (const foo = function(){}) is NOT hoisted -- must be defined before use.*

Q2. What is a closure and give a real use case?

A: *A function that remembers variables from its outer scope after the outer function returns. Use cases: private variables (counter), function factories (multiplier), memoization.*

Q3. What is the difference between rest and spread?

A: *Both use ... but opposites. Rest COLLECTS multiple values into array in function params. Spread EXPANDS array/object into individual elements when calling functions or creating new arrays/objects.*

Q4. What does .reduce() do?

A: *Collapses array to single value by applying function with accumulator. Use for: summing, finding max/min, grouping, converting array to object.*

Q5. What is 'this' inside an arrow function?

A: *Arrow functions have NO own 'this'. They inherit 'this' from the enclosing lexical scope (where the arrow is defined). This is why arrows are preferred for callbacks inside class methods.*

■ Functions & Scope Cheat Sheet	
`function name() {}`	Declaration -- hoisted
`const fn = () => {}`	Arrow -- no own 'this'
`fn(a, b=10)`	Default parameter
`fn(...args)`	Rest -- collects into array
`Math.max(...arr)`	Spread -- expands array
`const {a,b} = obj`	Object destructuring
`const [x,y] = arr`	Array destructuring
`[...arr1,...arr2]`	Merge arrays
`{...obj1,...obj2}`	Merge objects
`arr.map(fn)`	Transform every element
`arr.filter(fn)`	Keep matching elements
`arr.reduce(fn,0)`	Collapse to single value